

1 NeRF, 3DGS và hợp nhất cảm biến (Neural Rendering and Sensor Fusion)

PHỤ THUỘC C/16-thị-giac-3d/01-bon-the-he-3d (thể hệ ba nằm ở đầu trong pipeline); C/16-thị-giac-3d/02-bieu-dien-3d-va-depth (trường ẩn và Gaussian trong bảng biểu diễn); A/02-vat-ly-cao-anh (phơi sáng, cân bằng trắng, hiệu chỉnh ngoại).

TÓM TẮT

Mục này giải thích cơ chế thật của *neural rendering*: vì sao NeRF phải biểu diễn cảnh bằng mật độ liên tục chứ không bằng bề mặt, và 3DGS đối cái gì lấy tốc độ. Nửa sau chuyển sang bài toán hợp nhất camera–LiDAR–IMU. Đọc xong bạn quyết định được khi nào một bản đồ render được là đáng giá cho robot, và khi nào bản đồ thừa cổ điển vẫn đúng hơn. Bạn cũng tính được bằng con số vì sao *early fusion* nhạy với lệch hiệu chỉnh còn *late fusion* thì không. Nếu chỉ nhớ một điều: hàm mất mát của neural rendering chỉ ràng buộc *ảnh render*, không ràng buộc *hình học*. Nên PSNR cao không chứng minh bản đồ đúng. Nếu robot sắp va chạm dựa trên bản đồ đó, phải đánh giá bằng độ chính xác hình học so với một quét chân trị.

KIẾN THỨC CẦN CÓ TRƯỚC

- **novel view synthesis (dựng ảnh ở góc nhìn mới)** — bài toán: cho một số ảnh đã chụp của một cảnh, hãy tạo ra ảnh của cảnh đó từ một vị trí camera chưa từng chụp.
- **radiance field (trường bức xạ)** — một hàm gán cho mỗi điểm trong không gian và mỗi hướng nhìn một màu và một độ đặc; đây là “cảnh” mà NeRF học, thay cho bề mặt.
- **volume rendering (dựng ảnh thể tích)** — cách tổng hợp một pixel bằng cách lấy mẫu nhiều điểm dọc theo tia nhìn rồi cộng dồn màu có trọng số theo độ đục tích lũy — thay vì tìm giao điểm với một bề mặt cứng.
- **splat / Gaussian splatting** — “splat” là vết loang một hạt để lại khi bị chiếu xuống mặt ảnh. 3DGS mô tả cảnh bằng hàng triệu hạt Gaussian mờ, và render là chiếu chúng xuống rồi trộn theo thứ tự độ sâu.
- **rasterize** — chuyển một hình học đã chiếu thành pixel bằng cách quét trực tiếp trên lưới ảnh; nhanh hơn nhiều so với đi dò dọc từng tia, và là thứ GPU sinh ra để làm.
- **PSNR** — chỉ số đo sai lệch ảnh dựa trên bình phương trung bình theo pixel, tính bằng decibel; cao là “gần ảnh gốc theo nghĩa số học”, không phải “trông giống theo nghĩa thị giác”.
- **hiệu chỉnh ngoại (extrinsic calibration)** — phép xác định vị trí và hướng tương đối giữa hai cảm biến; sai một độ ở đây biến thành sai vài chục centimet ở tầm hai chục mét.
- **early / late fusion** — hai tầng hợp nhất cảm biến: gộp ở mức đặc trưng (sớm, giàu thông tin) hoặc gộp ở mức kết quả từng nhánh (muộn, bền hơn).

A. Đặt vấn đề

Giả sử SfM đã chạy xong: bạn có pose của bốn mươi tám ảnh và một point cloud thừa. Bây giờ hãy render cảnh đó từ một góc nhìn chưa từng chụp. Kết quả sẽ tệ, và tệ theo cách rất đặc trưng: thủng lỗ ở chỗ không có điểm, viền răng cưa ở biên vật thể, mặt kính và mặt kim loại sai hoàn toàn vì màu của chúng phụ thuộc góc nhìn còn point cloud chỉ lưu được một màu cho mỗi điểm. Vấn đề nằm ở chỗ pipeline cổ điển tối ưu cho *độ chính xác hình học* rồi hy vọng ảnh đẹp theo sau, trong khi cái ta cần ở đây là ngược lại.

Neural rendering đảo ngược thứ tự đó bằng một ý tưởng đơn giản tới mức khó chịu: *nếu thứ ta muốn là ảnh, hãy đặt ảnh vào hàm mất mát*. Tham số hoá cảnh bằng một biểu diễn nào đó, viết một bộ render **khả vi** (*differentiable*) từ biểu diễn đó ra ảnh, rồi hạ **gradient** trên tham số của biểu diễn cho tới khi ảnh render trùng khít mọi ảnh quan sát. Không có bước tìm tương ứng, không tam

giác hoá, không ràng buộc epipolar nào được viết ra — hình học nếu có xuất hiện thì xuất hiện như một *tác dụng phụ* của việc khớp ảnh. Câu “nếu có” đó là toàn bộ nội dung mục này.

Học mục này thì làm được gì mà trước đó không làm được? Ba việc rất cụ thể: đánh giá được một đề xuất kiểu “thay bản đồ SLAM bằng Gaussian” thay vì phải tin hay không tin; đọc được một bài neural rendering và chỉ ra ngay giả định nào trong ba giả định nền của nó bị dữ liệu robot vi phạm; và biến câu “hiệu chỉnh phải chính xác” thành một ngân sách sai số tính bằng centimet ở một tầm xa cụ thể.

TRỰC GIÁC

Vì sao NeRF dựng khối chứ không dựng tường? Vì đạo hàm. Nếu cảnh gồm các bề mặt cứng, dịch một bề mặt đi một phần nghìn mét thì một pixel hoặc đổi màu hẳn, hoặc không đổi gì cả. Hàm mất mát khi đó là bậc thang: gradient bằng không gần như mọi nơi, và **hạ gradient** (*gradient descent*) không biết vận về hướng nào. Thay bề mặt cứng bằng một trường mật độ liên tục thì mỗi pixel trở thành tổng có trọng số của hàng chục điểm dọc theo tia, và *mọi* điểm đều nhận được một phần gradient. Đây là phép đổi một công tắc bật/tắt thành một chiết áp: chỉ chiết áp mới nói được nên vận theo chiều nào.

B. NeRF: cảnh là một hàm, ảnh là tích phân dọc tia

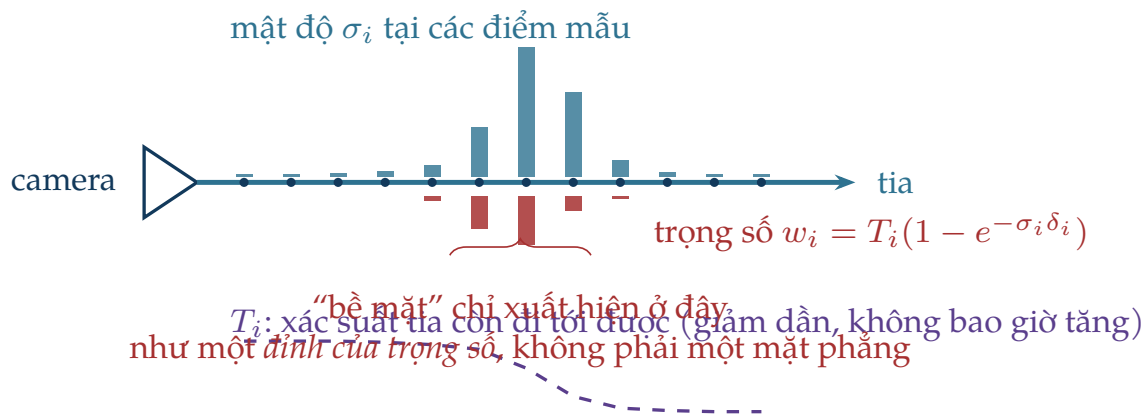
NeRF biểu diễn cảnh bằng một MLP nhận toạ độ 3D và hướng nhìn, trả về mật độ σ và màu c [1]. Để render một pixel, lấy mẫu N điểm dọc tia đi qua pixel đó rồi tổng hợp theo công thức dựng ảnh thể tích rồi raster hoá:

$$C(\mathbf{r}) = \sum_{i=1}^N T_i (1 - e^{-\sigma_i \delta_i}) c_i, \quad T_i = \exp\left(-\sum_{j<i} \sigma_j \delta_j\right).$$

Công thức này thực sự nói gì: T_i là xác suất tia đi tới được điểm thứ i mà chưa bị chặn, còn $1 - e^{-\sigma_i \delta_i}$ là xác suất nó dừng lại đúng ở đó; vậy màu pixel là kỳ vọng của màu tại điểm tia dừng lại. Điều đáng nhớ là mọi số hạng đều khả vi theo σ và c , nên gradient của sai số ảnh chảy ngược được về mọi điểm mẫu — đó là toàn bộ lý do NeRF hoạt động; phần MLP chỉ là một lựa chọn triển khai, và mục sau sẽ thay nó.

↪ **tiền đề** C/12/01 — toàn bộ cơ chế đứng trên một điều duy nhất: mọi nút trên đường từ tham số tới ảnh phải khả vi.

Hình 1 vẽ đúng ba đại lượng trong công thức trên một tia duy nhất.

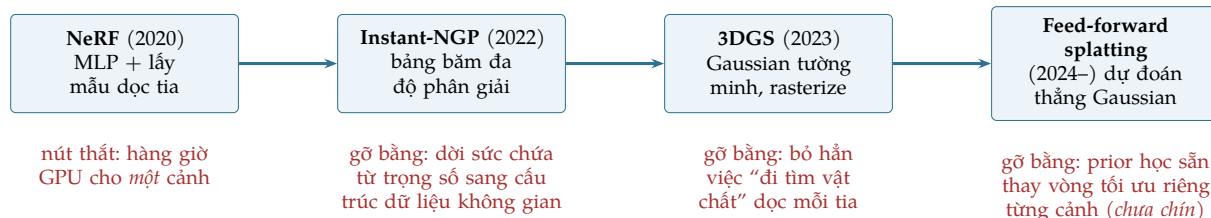


Hình 1. Dựng ảnh thể tích, vẽ trên một tia. Mạng không trả lời câu hỏi “mặt ở đâu” mà trả lời “mật độ tại điểm này bao nhiêu” ở hàng chục điểm dọc tia; màu pixel là tổng có trọng số của màu tại các điểm đó. Ba hàng trong hình giải thích công thức: mật độ σ_i quyết định tia dừng lại bao nhiêu ở mỗi điểm, T_i là phần tia còn sống sót tới đó, và tích của hai thứ cho trọng số. Vì mọi số hạng đều liên tục, mọi điểm mẫu đều nhận được gradient — đó chính là điều mà một bề mặt cứng không làm được.

Một chi tiết dễ bỏ qua nhưng quyết định chất lượng là **mã hoá vị trí** (*positional encoding*): nhồi thẳng tọa độ vào MLP cho ra ảnh mờ nhoè, vì MLP có thiên lệch mạnh về hàm tần số thấp. Ánh xạ tọa độ qua một dãy sin/cos nhiều tần số trước khi đưa vào mạng là cách trả lại cho mạng khả năng biểu diễn chi tiết sắc nét — cùng câu chuyện *spectral bias* đã gặp ở B/10-thiet-ke-kien-truc.

↪ **hệ quả của B/10** — mã hoá vị trí là cách bù cho một thiên lệch quy nạp có sẵn của MLP, chứ không phải một mẹo kỹ thuật rời rạc.

Cái giá là chi phí tính toán, và nó lớn hơn trực giác nhiều. Một ảnh 800×600 có 480 000 tia; với 64 điểm mẫu thô cộng 128 điểm mẫu tinh, ta cần khoảng $9,2 \times 10^7$ lần chạy MLP cho *một* khung hình. Nhân với hàng chục nghìn bước huấn luyện thì thành nhiều giờ GPU cho một cảnh duy nhất — và đây là nút thắt sinh ra cả dòng công trình tiếp theo, tóm tắt ở Hình 2.



Hình 2. Dòng thời gian của neural rendering, đọc theo nút thắt. Mỗi bước không phải “phiên bản tốt hơn” mà là câu trả lời cho đúng một nút cổ chai của bước trước; biết nút thắt nào đang cản trở bài toán của mình thì biết nên dừng ở mốc nào.

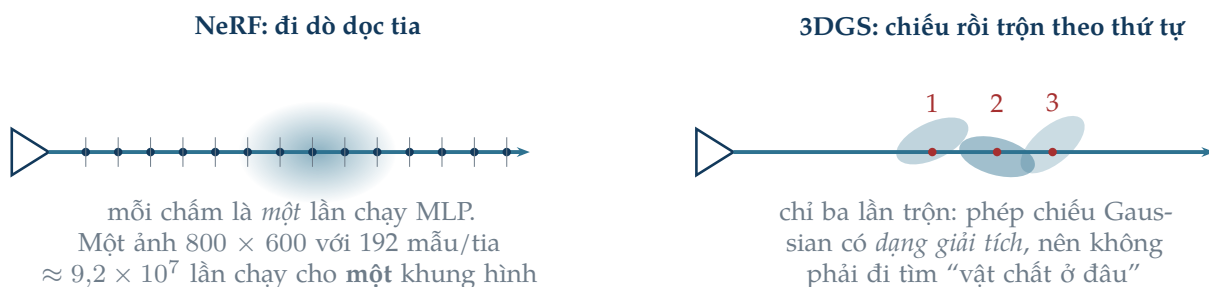
Instant-NGP tấn công đúng chi phí đó. Nó chuyển phần lớn sức chứa của mô hình từ trọng số MLP sang một bảng băm đa độ phân giải đặt trong không gian, để MLP chỉ còn nhỏ xíu [2]. Thời gian huấn luyện tụt từ hàng giờ xuống hàng chục giây. Đây là một mẫu hình sẽ gặp lại nhiều lần: khi mạng phải học một hàm có cấu trúc không gian rõ ràng, đưa cấu trúc đó vào *cấu trúc dữ liệu* rẻ hơn nhiều so với bất trọng số học lại nó.

C. 3DGS: đổi biểu diễn ẩn lấy biểu diễn tường minh

Ngay cả với bảng băm, NeRF vẫn phải đi dọc từng tia và lấy mẫu — vẫn “đi tìm vật chất ở đâu” cho mỗi pixel mỗi khung hình. 3D Gaussian splatting bỏ hẳn bước tìm kiếm đó: cảnh là một tập hữu hạn Gaussian dị hướng, mỗi cái có vị trí, ma trận hiệp phương sai, độ đục và màu phụ thuộc góc nhìn; render là chiếu từng Gaussian xuống mặt phẳng ảnh, sắp xếp theo độ sâu, rồi trộn theo

thứ tự [3]. Toán tổng hợp màu vẫn đúng công thức trên, nhưng phép chiếu có dạng giải tích nên bộ rasterize chạy được ở tốc độ thời gian thực.

Hình 3 đặt hai cách tính cùng một pixel cạnh nhau, và cho thấy khoản tiết kiệm đến từ đâu.



Hình 3. Cùng một tia, hai cách tính màu pixel. Bên trái, NeRF phải lấy mẫu dày dọc tia vì nó *không biết* trước vật chất nằm ở đâu — toàn bộ chi phí đến từ việc đi tìm. Bên phải, 3DGS đã lưu sẵn vật chất thành các nguyên thể tường minh, nên chỉ cần chiếu chúng xuống mặt ảnh, sắp xếp theo độ sâu và trộn ba lần. Toán tổng hợp màu ở hai bên là một; thứ thay đổi là ai đi tìm — và đó là toàn bộ khoản chênh hai tới ba bậc độ lớn về tốc độ render.

Bảng 1. Ba mốc của neural rendering. Cả ba chia chung một tập giả định — pose đã biết, cảnh tĩnh, phơi sáng nhất quán — nên hồng chung ở những chỗ đó, và hồng riêng ở cột cuối.

Phương pháp	Cơ chế	Chi phí	Hồng khi / mất gì
NeRF (MLP)	MLP + lấy mẫu dọc tia + tích phân thể tích	Huấn luyện hàng giờ; render < 1 fps; mô hình vài MB	Chậm cả hai đầu; không chỉnh sửa được cục bộ
Instant-NGP	Bảng băm đa độ phân giải thay phần lớn trọng số	Huấn luyện hàng chục giây; tốn VRAM	Va chạm băm gây nhiễu ở vùng chi tiết
3D Gaussian splatting	Tập Gaussian tường minh, rasterize có sắp xếp	Render thời gian thực; mô hình vài trăm MB	Dung lượng lớn; bề mặt không tường minh; nhạy với nhiễu tạo từ SfM

Tính tay dung lượng một lần cho nhớ. Mỗi Gaussian lưu 3 số vị trí, 3 số tỉ lệ, 4 số quaternion, 1 số độ đục và 48 hệ số **hàm điều hoà cầu** (*spherical harmonics*) bậc ba cho màu: tổng 59 số thực 32-bit, tức 236 byte. Một cảnh phòng khách dựng tốt thường cần cỡ một triệu Gaussian, tức khoảng 236 MB — so với vài megabyte cho trọng số của một NeRF MLP. Đó chính là “giá phải trả” hay được nhắc chung chung: đổi khoảng hai bậc độ lớn dung lượng lấy khoảng hai tới ba bậc độ lớn tốc độ render.

Cái mất thứ hai tinh tế hơn và quan trọng hơn với người làm robot: **hình học kém tường minh**. Trong một trường *SDF*, bề mặt là tập mức 0 — định nghĩa sạch, trích mesh bằng *marching cubes* là chuyện cơ học. Trong 3DGS, “bề mặt” không được định nghĩa ở đâu cả: tối ưu chỉ đòi ảnh render đúng, nên nó thoải mái đặt các Gaussian dẹt lơ lửng trước một bức tường nếu làm vậy khớp ảnh tốt hơn. Muốn trích mesh dùng được, người ta phải thêm ràng buộc ép Gaussian bám bề mặt (dẹt hoá, gấn pháp tuyến) — tức là trả lại đúng phần ràng buộc hình học mà bước đầu đã cố tình bỏ đi.

D. Ba giả định chung, và điều gì xảy ra khi chúng sai

Cả họ neural rendering đứng trên ba giả định, và với dữ liệu quay bằng camera robot thì cả ba đều dễ vỡ:

- **Pose đã biết và chính xác.** Sai lệch vài phần mười độ làm các ảnh mâu thuẫn nhau; vì tối ưu luôn tìm cách giảm loss, nó bù bằng cách sinh ra vật chất giả lơ lửng — hiện tượng *floater*. Đây là lý do 3DGS “nhảy với khối tạo từ SfM”: rác vào thì rác ra, chỉ khác là rác ra trông rất đẹp.
- **Cảnh tĩnh.** Người đi qua khung hình để lại vệt mờ hoặc khối đục lơ lửng ở giữa phòng.
- **Quá trình tạo ảnh nhất quán.** Camera bật phơi sáng và cân bằng trắng tự động vì phạm giả định này liên tục, và mô hình buộc phải giải thích thay đổi độ sáng bằng cách nhét thêm hình học sai. **Ảnh mờ do chuyển động** (*motion blur*) cũng thuộc nhóm này: một khung hình mờ không phải quan sát nhiều của cảnh tĩnh mà là tích phân của cảnh dọc một quãng chuyển động, và mô hình render không có chỗ nào biểu diễn điều đó.

CẠM BẮY

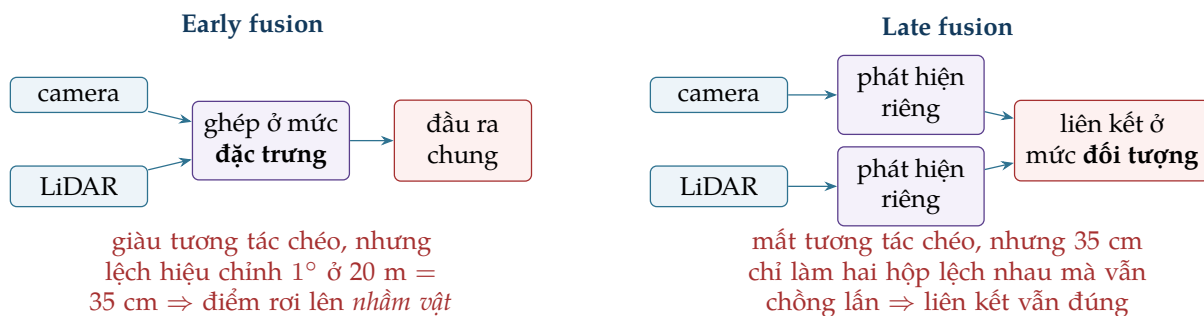
PSNR cao trên các view giữ lại *không* chứng minh hình học đúng. Hàm mất mát chỉ ràng buộc ảnh render; hai cấu hình mật độ rất khác nhau có thể sinh ra cùng bộ ảnh, đặc biệt khi màu được phép phụ thuộc hướng nhìn — mô hình có thể “vẽ” thứ đáng lẽ phải “dựng”. Triệu chứng chẩn đoán: chất lượng đẹp ở các view nội suy gần quỹ đạo chụp và sập ngay khi ngoại suy ra ngoài. Nếu bạn định dùng đầu ra làm bản đồ để robot va chạm hay bám bề mặt, hãy đánh giá bằng độ chính xác hình học so với một quét chân trị, không phải bằng PSNR hay *LPIPS* [4].

Với người làm SLAM, hệ quả thực hành gọn lại thành một câu hỏi: *bạn có thực sự cần render không?* Nếu chỉ cần định vị, bản đồ thừa của SLAM cổ điển vẫn là lựa chọn đúng — rẻ hơn nhiều bậc và có sai số kiểm chứng được. Neural rendering đáng đưa vào khi bài toán hạ nguồn thật sự cần ảnh: kiểm tra trực quan từ xa, sinh dữ liệu huấn luyện từ góc nhìn mới, mô phỏng cảm biến, hoặc định vị lại bằng cách so ảnh render với ảnh thật.

→ cùng nguyên lý với C/17/02 — cùng một ý tưởng “đặt thứ ta muốn vào hàm mất mát rồi tối ưu trên chính đối tượng cần tạo ra”, chỉ khác biến tối ưu là ảnh thay vì cảnh.

E. Hợp nhất camera, LiDAR và IMU

Ba cảm biến này bù trừ cho nhau gần như hoàn hảo: camera cho ngữ nghĩa và độ phân giải góc cao nhưng không có tỉ lệ; LiDAR cho hình học metric thừa và bền với ánh sáng; IMU cho chuyển động tần số cao và quan sát được phương trọng lực nhưng trôi rất nhanh khi tích phân. Câu hỏi thiết kế là hợp nhất ở tầng nào (Hình 4).



Hình 4. Cùng một sai số hiệu chỉnh, hai hậu quả khác hẳn nhau. Early fusion liên kết ở mức pixel nên sai số biến thành *đầu vào rác*; late fusion liên kết ở mức đối tượng, nơi vài chục centimet vẫn nằm trong kích thước của vật, nên sai số chỉ thành nhiễu đầu ra. Đó là toàn bộ nội dung của cuộc đánh đổi “giàu thông tin” đổi lại “bền vững”.

Con số trong hình tính ra như sau: sai số hiệu chỉnh ngoại xoay 1° chuyển thành sai số ngang $\tan 1^\circ \approx 17,5$ mm trên mỗi mét tầm xa, nên ở 20 m là 35 cm. Lệch đồng bộ thời gian sinh ra đúng loại lỗi đó, nhưng âm thầm hơn vì nó chỉ xuất hiện khi có chuyển động. Robot quay $100^\circ/\text{s}$ với lệch nhãn thời gian 5 ms cho sai lệch góc $0,5^\circ$. Con số này tỉ lệ thuận với tốc độ quay. Nghĩa là hệ thống chạy hoàn hảo khi đứng yên, rồi hỏng đúng lúc đang xoay nhanh — đúng lúc bạn cần nó nhất. Đây là lý do một nhận định lặp đi lặp lại trong thực hành: ở các hệ đa cảm biến, phần lớn thời gian sửa lỗi nằm ở đồng bộ thời gian và hiệu chỉnh ngoại chứ không ở thuật toán hợp nhất.

CẦN NHỚ

Sai số hiệu chỉnh và sai số đồng bộ không phải nhiễu — chúng là sai số *có hệ thống*, phụ thuộc tầm xa và tốc độ, nên không trung bình về không khi gom nhiều quan sát. Hệ quả bất buộc: chúng phải được đưa vào mô hình như biến cần ước lượng (hiệu chỉnh trực tuyến, ước lượng độ trễ t_d) chứ không phải được hy vọng là nhỏ. Và với neural rendering, điều tương đương là: hàm mất mát ràng buộc ảnh, nên hình học phải được kiểm tra bằng một phép đo bên ngoài. [T4]

F. Giới hạn và trường hợp hỏng

Bên cạnh bài toán tối ưu riêng từng cảnh (per-scene optimization), người triển khai 3DGS và NeRF trên robot và hệ thống nhúng phải đối mặt với ba rủi ro kỹ thuật rất cụ thể:

- **Bùng nổ bộ nhớ VRAM khi mở rộng bản đồ (VRAM Explosion):** Một phòng làm việc nhỏ có thể sinh ra từ 2 tới 5 triệu hạt Gaussian, ngốn từ 1.5 tới 3 GB VRAM trên GPU. Khi robot di chuyển qua toà nhà hoặc môi trường ngoài trời rộng lớn, việc lưu trữ toàn bộ Gaussian trong bộ nhớ đồ hoạ dẫn tới tràn RAM GPU (OOM) lập tức. Mặc dù các kỹ thuật nén (như Scaffold-GS hay lượng tử hoá thuộc tính) giảm dung lượng đáng kể, chúng làm tăng chi phí giải nén thời gian thực và làm mờ các chi tiết hình học mảnh.
- **Vật cản ma (Phantom Obstacles) và hình học suy biến:** Do hàm mất mát chỉ tối ưu trên ảnh 2D, tại những vùng camera ít quan sát hoặc góc nhìn hẹp, 3DGS có xu hướng sinh ra các Gaussian hình cây kim dài (needle-like Gaussians) hoặc các đám mây lơ lửng trước ống kính nhằm hạ loss một cách cơ hội. Đối với một hệ thống robot tự hành lấy bản đồ này để lập đường đi, những đám mây giả đó sẽ bị hiểu nhầm là vật cản thực tế, làm robot dừng đột ngột hoặc lặp lộ trình vòng vèo vô ích.
- **Tính phi bất biến trước biến thiên ánh sáng:** Cả NeRF và 3DGS sử dụng hàm cầu điều hoà (Spherical Harmonics) để mô tả tính phản xạ phụ thuộc hướng nhìn (view-dependent appearance). Khi môi trường có ánh sáng biến động mạnh (đèn bật/tắt, bóng râm thay đổi, phơi sáng cam-

era tự động nhảy), mô hình sẽ nhầm lẫn giữa sự thay đổi bức xạ và sự thay đổi hình học, từ đó “khắc” các vết sáng tối vĩnh viễn vào vị trí của các hạt 3D.

Về hợp nhất, chế độ hỏng đáng sợ nhất là hỏng *im lặng* của một nhánh. Mưa làm LiDAR sinh điểm giả; đường hầm làm camera mất hết đặc trưng; rung động làm IMU bão hoà. Một hệ hợp nhất không kiểm tra tính nhất quán giữa các nhánh sẽ vẫn cho ra ước lượng trông bình thường, với hiệp phương sai cũng trông bình thường. Vì vậy phần đáng đầu tư trong một hệ thật thường không phải bộ hợp nhất mà là bộ chẩn đoán từng nhánh: phần dư theo từng cảm biến, kiểm tra chi bình phương, và một cơ chế hạ cấp có kiểm soát khi một nhánh bị loại.

↪ **hệ quả của E/25** — hỏng im lặng của một nhánh cảm biến là trường hợp cụ thể của yêu cầu chung: hệ thống phải biết lúc nào nó không đáng tin.

G. Kết nối

Mục này khép lại chương 16 và nối ngược lên C/16-thi-giac-3d/01-bon-the-he-3d: neural rendering là thể hệ ba, đứng *sau* khâu pose, nên nó tiêu thụ đầu ra của thể hệ một chứ không thay thế. Ý tưởng “đặt cái ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra” sẽ trở lại nguyên vẹn ở C/17-mo-hinh-sinh/02-toi-uu-tren-pixel, chỉ khác là biến tối ưu ở đó là ảnh chứ không phải cảnh. Các giả định về quá trình tạo ảnh thuộc A/02-vat-ly-tao-anh; việc chẩn đoán từng nhánh cảm biến và hạ cấp có kiểm soát thuộc E/25-do-tin-cay-va-van-hanh.

▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao biểu diễn cảnh bằng mật độ liên tục là điều kiện cần để gradient descent hoạt động, trong khi bề mặt cứng thì không?
2. 3DGS nhanh hơn NeRF nhờ đổi cái gì lấy cái gì, và vì sao cái mất đó lại đặc biệt bất tiện cho robot?
3. Một cảnh có PSNR rất cao trên các view giữ lại nhưng bản đồ hình học sai. Cơ chế nào cho phép hai điều đó cùng đúng, và bạn thiết kế phép thử nào để phát hiện?
4. Vì sao một hệ early fusion có thể chạy hoàn hảo khi robot đứng yên và hỏng khi robot xoay nhanh, dù không tham số nào thay đổi?
5. Nếu buộc phải chọn giữa một tuần cải tiến thuật toán hợp nhất và một tuần dựng lại quy trình hiệu chỉnh cùng đồng bộ thời gian, bạn chọn cái nào và dựa trên bằng chứng gì?
6. Hình 2 đọc theo nút thắt. Trong bài toán của bạn, nút thắt đang cản là thời gian dựng, tốc độ render, hay dung lượng bản đồ — và điều đó chỉ về mốc nào?

Nguồn

- [1] B. Mildenhall, P. P. Srinivasan, M. Tancik, et al., “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis,” in *ECCV*, 2020.
- [2] T. Müller, A. Evans, C. Schied, et al., “Instant Neural Graphics Primitives with a Multiresolution Hash Encoding,” *ACM TOG*, vol. 41, no. 4, 2022.
- [3] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [4] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.